

01 / EMBEDDED SOFTWARE

Arduino Programming

| From Bare-Metal C/C++ Logic to Autonomous Hardware Control.

ACTUATOR 26

FIELD NOTES / 2026



02 / SKETCH ARCHITECTURE

The Execution Model

setup() & loop()

Every Arduino sketch relies on two core lifecycle functions. The microcontroller initializes hardware once, then enters an infinite execution loop.

STEP 1 · RUNS ONCE

setup()

Configures GPIO pin directions, sets up serial communications, initializes sensors, and allocates buffers on power-up.

STEP 2 · REPEATS FOREVER

loop()

Contains main execution logic. Continuously evaluates conditions, reads sensors, and updates outputs until powered off.

03 / SKETCH ARCHITECTURE

Standard Sketch Structure Code

The fundamental skeleton of every embedded C++ application:

```
// Standard Arduino Sketch Skeleton

void setup() {
  pinMode(LED_BUILTIN, OUTPUT); // Configure LED output pin
  Serial.begin(115200);         // Start serial UART at 115200 baud
}

void loop() {
  digitalWrite(LED_BUILTIN, HIGH); // Turn LED ON
  delay(500);                       // Wait 500ms
  digitalWrite(LED_BUILTIN, LOW);  // Turn LED OFF
  delay(500);                       // Wait 500ms
}
```

04 / DATA TYPES & RAM

Choosing the Right Data Type

Microcontrollers have limited SRAM. Selecting optimal variable types saves precious RAM.

TYPE	SIZE	VALUE RANGE	BEST USED FOR
bool	1 byte	true (1) or false (0)	Digital states, flag conditions
uint8_t / byte	1 byte	0 to 255	Pin IDs, 8-bit PWM values
int	2 bytes	-32,768 to 32,767	Sensor counts, general math
unsigned long	4 bytes	0 to 4,294,967,295	Timers (‘millis()’, ‘micros()’)
float	4 bytes	±3.4e38 (approx 6 decimals)	Analog calculations, angles

05 / DIGITAL HARDWARE

Reading & Writing Digital Signals

Core functions to interface with digital sensors, switches, and relays.

CONFIGURE**`pinMode(pin, mode)`**

Sets GPIO pin direction to INPUT, OUTPUT, or INPUT_PULLUP.

WRITE**`digitalWrite(pin, state)`**

Drives output pin to HIGH (3.3V/5V) or LOW (0V).

READ**`digitalRead(pin)`**

Reads logical level (HIGH or LOW) of a pin.

PRO TIP**INPUT_PULLUP**

Engages internal resistor. Keeps pin HIGH until connected to ground by a switch.

06 / DIGITAL HARDWARE

Digital Reading & Control Code

Reading a push-button with internal pull-up and controlling an output LED:

```
const int BUTTON_PIN = 2;
const int LED_PIN = 13;

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP); // Active-LOW button
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  int buttonState = digitalRead(BUTTON_PIN);
  if (buttonState == LOW) {
    digitalWrite(LED_PIN, HIGH); // Button Pressed -> LED ON
  } else {
    digitalWrite(LED_PIN, LOW); // Released -> LED OFF
  }
}
```

07 / ANALOG SIGNALS

Analog Inputs & ADC Conversion

Converts continuous real-world voltages into digital readings (0–1023 for 10-bit ADC).

$$V_{in} = \frac{\text{ADC Reading} \times V_{ref}}{1023}$$

For 5V reference: $V_{in} = \text{rawValue} * (5.0 / 1023.0)$

```
// Read Voltage from Potentiometer

const int POT_PIN = A0;

void setup() {
  Serial.begin(115200);
}

void loop() {
  int rawValue = analogRead(POT_PIN);
  float voltage = rawValue * (5.0 / 1023.0);
  Serial.print("Voltage: ");
  Serial.println(voltage);
  delay(100);
}
```

08 / PWM OUTPUT

Pulse-Width Modulation

Speed & Dimming

Simulating analog output using high-frequency digital pulses.

- ↳ **8-Bit Resolution**
Accepts values from 0 (0% Duty) to 255 (100% Duty).
- ↳ **Hardware Control**
DC motor speed control, LED dimming, buzzer tones.

```
// Fade LED brightness / Motor speed with PWM
```

```
const int PWM_PIN = 9;
```

```
void setup() {  
  pinMode(PWM_PIN, OUTPUT);  
}
```

```
void loop() {  
  analogWrite(PWM_PIN, 128); // 50% Duty Cycle  
}
```

09 / REAL-TIME TIMING

Ditch `delay()` Master `millis()`

THE HAZARD

`delay()` Freezes the CPU

While `delay(1000)` runs, the microcontroller is locked. It CANNOT sample sensors, process commands, or trigger emergency stops!

THE SOLUTION

`millis()` Non-Blocking Timer

Returns elapsed milliseconds since power-on. Compare timestamps to run concurrent tasks smoothly.

10 / REAL-TIME TIMING

Non-Blocking Multitasking Timer Code

Executing periodic tasks without blocking CPU responsiveness:

```
// Blink LED every 500ms without delay()

unsigned long prevMillis = 0;
const long INTERVAL = 500; // Time interval in ms
bool ledState = LOW;

void loop() {
    unsigned long currentMillis = millis();

    if (currentMillis - prevMillis >= INTERVAL) {
        prevMillis = currentMillis;
        ledState = !ledState;
        digitalWrite(LED_PIN, ledState);
    }
    // Other tasks (sensors, motors) run continuously!
}
```

11 / UART COMMUNICATION

Talk to your Machine

Serial UART

Hardware serial communication for debugging, telemetry, and external control.

INIT**`Serial.begin(115200)`**

Initializes UART peripheral at specified baud rate (e.g. 9600, 115200).

SEND**`Serial.println(data)`**

Transmits ASCII data to Host PC or Bluetooth device with newline.

CHECK**`Serial.available()`**

Returns the number of unread bytes sitting in the RX buffer.

READ**`Serial.read()`**

Pulls the oldest single byte from the RX buffer.

12 / UART COMMUNICATION

Serial Command Receiver Code

Parsing incoming character commands from Serial monitor or Bluetooth:

```
// Serial Command Parsing Example

void setup() {
  Serial.begin(115200);
}

void loop() {
  if (Serial.available() > 0) {
    char command = Serial.read();
    if (command == 'F') {
      driveForward();
      Serial.println("STATUS: Moving Forward");
    } else if (command == 'S') {
      stopMotors();
      Serial.println("STATUS: Motors Stopped");
    }
  }
}
```

13 / ASYNCHRONOUS EVENTS

Hardware Interrupts

Zero-Latency Response

Instant hardware-triggered callbacks that preempt normal code execution.

TRIGGER MODES**RISING / FALLING / CHANGE**

Interrupt triggers immediately on pin signal transitions without polling in `loop()`.

KEYWORD**`volatile`**

Prevents C++ compiler from caching variables shared between ISR and main loop in registers.

14 / ASYNCHRONOUS EVENTS

Interrupt Service Routine (ISR) Code

Emergency Stop Button handled with zero latency:

```
const byte INTERRUPT_PIN = 2;
volatile bool emergencyStop = false;

void handleEmergency() { // ISR Callback - Must be short & fast!
    emergencyStop = true;
}

void setup() {
    pinMode(INTERRUPT_PIN, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(INTERRUPT_PIN), handleEmergency, FALLING);
}

void loop() {
    if (emergencyStop) {
        cutMotorPower();
    }
}
```

15 / MODULAR CODE

Custom Functions & Reference Passing

Organizing complex sketches into clean, reusable logic blocks.

PASS BY VALUE

```
func(int val)
```

Creates a local copy of parameter. Safe for basic configuration numbers.

PASS BY REFERENCE

```
func(int &val)
```

Passes memory address directly. Allows functions to modify variables in caller scope.

16 / MODULAR CODE

HC-SR04 Ultrasonic Sensor Module Code

Encapsulating pulse measurement logic inside a helper function:

```
// Distance Calculation Helper Function

int getDistanceCM(int trigPin, int echoPin) {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    long duration = pulseIn(echoPin, HIGH);
    return duration * 0.034 / 2; // Compute CM distance
}

void loop() {
    int distance = getDistanceCM(9, 10);
    if (distance < 15) {
        avoidObstacle();
    }
}
```

17 / HARDWARE INTERFACING

Actuators, Motors & Sensor Buses

MOTOR DRIVERS**L298N / TB6612**

H-Bridge drivers require direction control pins + PWM pins for speed regulation.

SERVO MOTORS**Servo.h**

Sends 50Hz PWM pulses (1ms–2ms) mapping directly to 0°–180° angular position.

I2C BUS**Wire.h (SDA / SCL)**

Addressable 2-wire serial bus for connecting OLED displays, IMUs, and RTC modules.

SPI BUS**SPI.h (MOSI / MISO / SCK)**

High-speed bus for fast TFT screens, SD card readers, and wireless transceivers.

18 / ENGINEERING RULES

4 Gold Rules of Embedded C++

RULE 1**Shared Ground**

Always connect the GND of your microcontroller, motor drivers, and external power supplies together!

RULE 2**Isolate Motor Supplies**

Never power DC motors or heavy servos from the Arduino 5V/3.3V pin. Current spikes cause immediate resets!

RULE 3**Level Shift 5V Signals**

Use resistor dividers or bidirectional level shifters when connecting 5V signals to 3.3V GPIOs (ESP32/ARM).

RULE 4**Avoid Dynamic Allocation**

Avoid String concatenation in `loop()` to prevent heap fragmentation. Use static `char[ ]` buffers.

19 / READY TO BUILD

Build. Flash. Deploy.

You now have a solid foundation in embedded software engineering for ACTUATOR 26 robotics projects.

ARDUINO · EMBEDDED C++ · ACTUATOR 26

LET'S
BUILD IT